

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

Software Design Document (SDD)

AI-POWERED INSURANCE SIMULATOR

Introduction to Software Engineering

Group Bồng

Nguyễn Hoàng Thái - 20235557

Đặng Xuân Đạt - 20235485

Phạm Lê Minh Quang - 20235554

Phạm Ngọc Thạch - 20235556

Nguyễn Thành Vinh - 20235576

Ha Noi, June 2026

TABLE OF CONTENTS

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	4
2. References	5
3. System Architectural Design	5
3.1 System Description	5
3.2 System Architecture	6
3.3 Design Constraints	7
3.3.1 General constraints	7
3.3.2 Hardware constraints	7
3.3.3 Software constraints	7
4. Components Description	8
4.1 Introduction	8
4.2 Decomposition Description	8
4.2.1 Backend components	9
4.2.2 API (router) components	9
4.2.3 Data components (domain entities)	10
4.2.4 Frontend components	11
5. External Interfaces	11
5.1 Introduction	11
5.2 User Interfaces	12
5.3 Internal REST API Interface	13
5.4 External System Interfaces	14
5.5 Database Interface	15
5.6 Communication, Security, and Error Interface	15
6. Detailed Design	16
6.1 Introduction	16
6.2 Component - Authentication and KYC Service (auth_service)	16
6.3 Component - Wallet Service (wallet_service)	17
6.4 Component - Insurance Product Service (insurance_service)	17
6.5 Component - Policy Purchase Service (policy_service)	18
6.6 Component - Risk and ML Engine (risk_engine + ml_models)	18
6.7 Component - Simulation Engine	19
6.8 Component - Parametric Claims Engine (claims_engine)	19
6.9 Component - External Trigger Monitor	20
6.10 Component - AI Chatbot Service	20
6.11 Component - Notification Service	21
6.12 Component - Admin and Monitoring	21

6.13 Cross-Cutting Design Decisions	22
7. Annexes	22
7.1 Requirements Traceability Matrix	22
7.2 Non-Functional Requirement Coverage	23
7.3 Endpoint-to-Component Mapping	24
7.4 Data and Relationship Summary	25
7.5 IEEE 1016 and UML Mapping	25

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture and detailed design of the AI-Powered Insurance Simulator. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS) into a concrete technical design that the development team can implement, test, and maintain.

The document presents the overall system architecture, the decomposition of the system into components, the internal and external interfaces, and the detailed behaviour of the most critical components. It is intended to be the bridge between the requirement (functional) view captured in the SRS and the structural (implementation) view embodied in the source code repository.

Source code reference: github.com/minh-quang-pham-le/Insurance-Simulator. The design described here reflects the implementation present in that repository.

1.2 Scope

The AI-Powered Insurance Simulator is a simplified fintech platform that teaches users the concepts of risk management and micro-insurance. Users browse and purchase short-term parametric policies (flight delay, crop/weather, gadget, natural-disaster and rainfall coverage) using a virtual currency (SimCoin) held in a virtual wallet. An automated parametric claims engine settles claims using real-world data triggers, an AI chatbot assists purchase decisions, and a risk engine estimates event probabilities from historical data.

The platform serves two audiences through two separate single-page applications backed by one REST API and one database: an end-user Web App and an administrator Dashboard. This SDD covers the design of the backend services, both frontends, the data model, and the integrations with external data providers. It does not cover real financial transactions; all currency and payouts are simulated for educational purposes.

Intended audience: project stakeholders, the product owner (course instructor), and the development team (designers, programmers and testers).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SDD	Software Design Document — this document.
SRS	Software Requirements Specification — the baseline requirement document.
IEEE	Institute of Electrical and Electronics Engineers (design standard IEEE 1016).
IAM / RBAC	Identity & Access Management / Role-Based Access Control — restricts access based on assigned roles (USER, ADMIN).
Parametric Insurance	A policy that pays a predefined amount when an objective, measurable trigger event occurs, bypassing manual loss assessment.

Term	Definition
Trigger Condition	A mathematically defined threshold embedded in a policy; when external data meets it, the claim evaluation starts.
External Data Provider	A third-party API supplying real-time objective data (e.g. OpenWeatherMap, AviationStack).
API Payload	The data packet returned by an External Data Provider, consumed by the claims engine.
Virtual Wallet / SimCoin (SC)	A simulated ledger and currency isolated within the platform; no real money is involved.
JWT	JSON Web Token — the access/refresh token format used for authentication.
SPA	Single-Page Application — the Vue 3 frontend architecture used by both clients.
ORM	Object-Relational Mapping — SQLAlchemy, used to map Python classes to PostgreSQL tables.

2. References

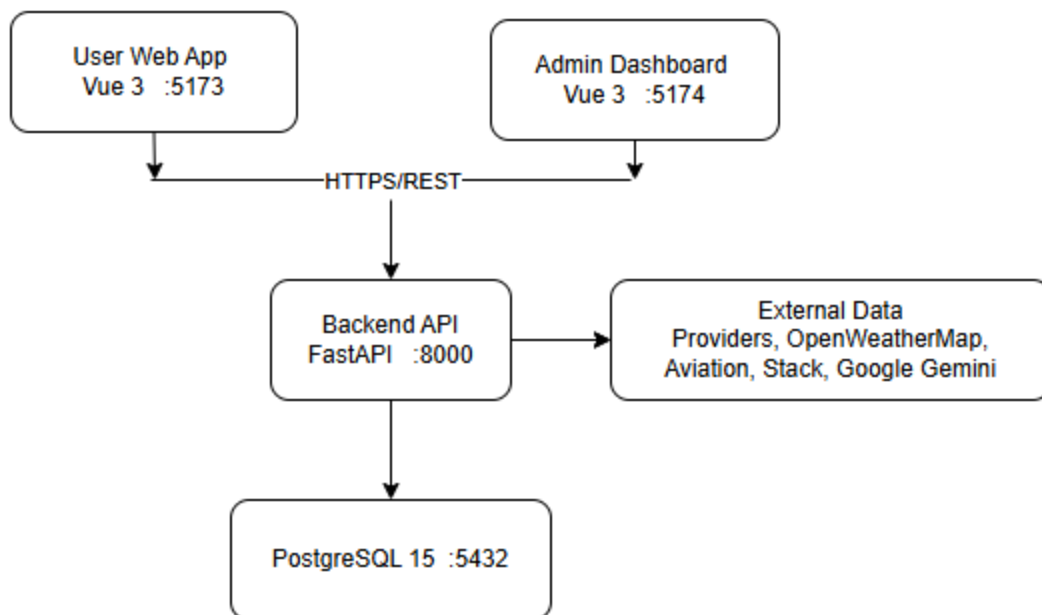
1. Software Requirements Specification (SRS) — AI-Powered Insurance Simulator, Version 1, Group Bồng, HUST, May 2026.
2. Project source repository — github.com/minh-quang-pham-le/Insurance-Simulator (README.md, SPEC.md, docs/ERD.md).
3. IEEE Std 1016, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
4. IEEE Std 1002-1987, IEEE Standard Taxonomy for Software Engineering Standards.
5. FastAPI documentation - <https://fastapi.tiangolo.com> ; SQLAlchemy 2.0 documentation - <https://docs.sqlalchemy.org> ; Vue 3 documentation - <https://vuejs.org>.

3. System Architectural Design

3.1 System Description

The system is a web-based platform composed of four cooperating runtime units: two browser-based client applications, one backend API server, and one relational database. The backend additionally integrates with three external services (a weather API, a flight-status API, and an LLM provider) that supply the data and intelligence behind the parametric and AI features.

At a high level, an end user interacts with the User Web App to browse products, run risk simulations, purchase policies, and manage a virtual wallet. An administrator uses the Admin Dashboard to manage products, review KYC, monitor claims, and oversee the parametric engine. Both clients call the same REST API, which encapsulates all business logic and is the only component that touches the database and the external providers.



3.2 System Architecture

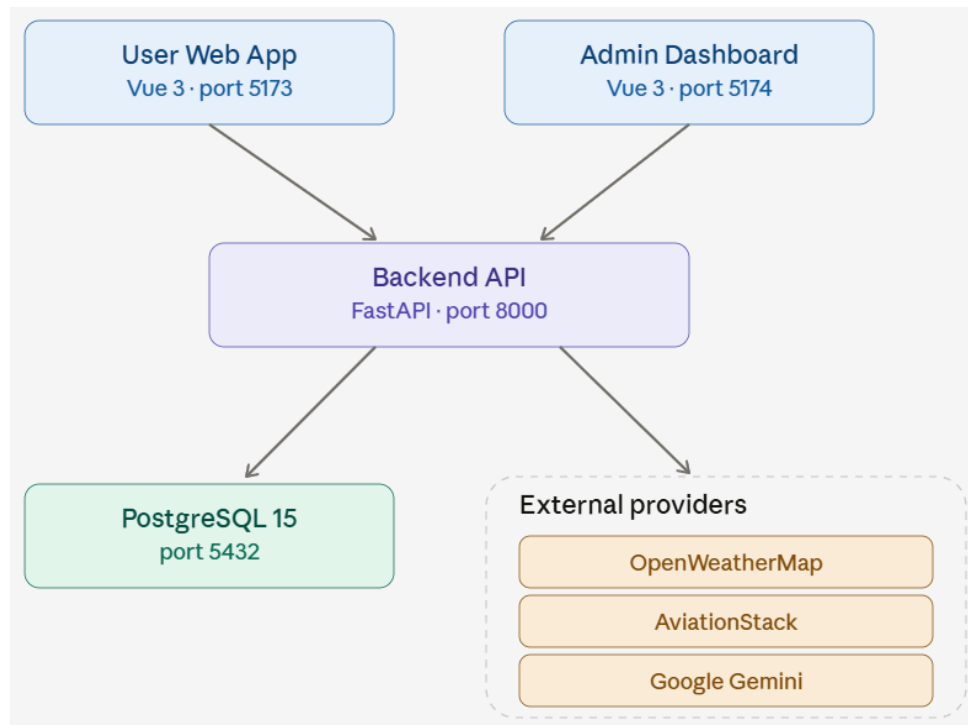
The platform follows a layered client/server architecture with a clear separation between presentation, application logic and data:

- **Presentation tier:** Two independent Vue 3 + Vite single-page applications (user and admin). Each is organised into views, reusable components, Pinia stores for state, and a thin service layer that wraps Axios HTTP calls to the API.
- **Application tier:** A FastAPI server organised in three layers — routers (HTTP endpoints and request/response validation via Pydantic schemas), services (business logic), and models (SQLAlchemy ORM entities). Cross-cutting concerns such as JWT authentication are handled by middleware.
- **Data tier:** PostgreSQL 15, accessed exclusively through the SQLAlchemy ORM; schema evolution is managed with Alembic migrations.
- **Integration tier:** Dedicated service modules call external providers (weather, flight status) and the Gemini LLM, isolating third-party concerns from core logic and recording calls in an API monitor log.

The architecture is deployed as containerised services orchestrated by Docker Compose. Each of the four runtime units (database, backend, user frontend, admin frontend) runs in its own container, which supports horizontal scalability of the stateless backend as required by the SRS.

Architectural rationale and alternatives: A layered monolithic API was chosen over a microservices architecture because the project scope, team size and timeline favour simplicity, a single shared data model, and straightforward transactional consistency (important for wallet operations). The router/service/model split keeps the codebase modular enough that components can later be extracted into separate services if needed. A purely server-rendered

approach was rejected in favour of SPAs to deliver the interactive simulation and dashboard experiences described in the SRS



3.3 Design Constraints

3.3.1 General constraints

- The project must be completed within the Introduction to Software Engineering course timeline by a small team, favouring well-known frameworks over custom infrastructure.
- All money, premiums and payouts are simulated (SimCoin); the system must carry no real-world financial or legal obligation.
- Critical operations (policy purchase, wallet credit after a claim) must satisfy ACID properties; failures must roll back to prevent inconsistent wallet states.

3.3.2 Hardware constraints

- The system targets commodity server/container hosts; no specialised hardware (GPU, etc.) is required. ML risk models are lightweight and run on CPU.
- Clients are standard desktop/laptop web browsers; no native mobile build is in scope.

3.3.3 Software constraints

- Backend: Python 3.10+, FastAPI, SQLAlchemy 2.0, Pydantic, Alembic. Database: PostgreSQL 15.
- Frontend: Vue 3, Vite, TailwindCSS, PrimeVue, ApexCharts, Pinia, Axios.
- AI / external services: Google Gemini 2.0 Flash (chatbot), OpenWeatherMap and AviationStack (trigger data); scikit-learn / joblib for the risk models.

- Infrastructure: Docker and Docker Compose. Authentication uses JWT access + refresh tokens; passwords are hashed before storage; external API keys are kept server-side only.

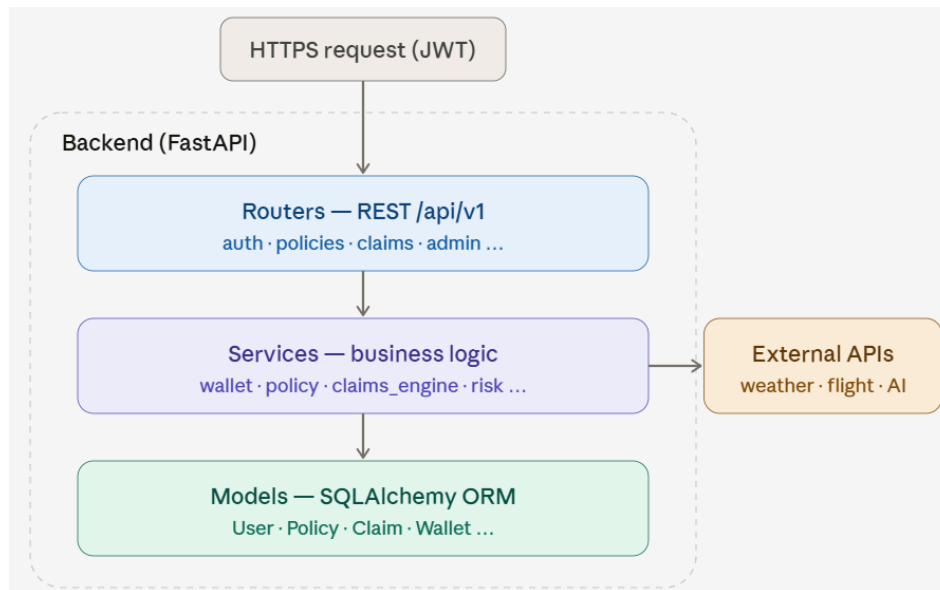
4. Components Description

4.1 Introduction

The system is decomposed following a layered, responsibility-driven strategy aligned with the use-case view of the SRS. The backend uses a Router–Service–Model pattern (a pragmatic variant of MVC where routers act as controllers, services hold domain logic, and models are the persistent domain entities). Each frontend follows the standard Vue component/store/service decomposition. The objective is that each component can be implemented, changed and tested with minimal impact on the others.

4.2 Decomposition Description

The platform is partitioned into the high-level subsystems below, each grouping a set of cohesive components



Subsystem	Responsibility
Identity & Access (IAM)	Registration, login, JWT issuing/refresh, KYC, RBAC enforcement (USER vs ADMIN).
Virtual Wallet	Balance management, top-up, premium deduction, payout credit, transaction ledger.
Marketplace & Policy	Product catalogue, premium calculation, policy purchase and lifecycle.
Simulation & Risk Engine	Risk-probability estimation from historical data and ML models; scenario simulation.

Subsystem	Responsibility
Parametric Claims Engine	Trigger evaluation, automatic and manual claim processing, payout orchestration.
AI Chatbot	LLM-backed product advisory using catalogue and risk context.
Notification	User notifications for claims, payouts and policy lifecycle events.
Admin & Monitoring	Product management, KYC review, claim oversight, external-API health monitoring, analytics.

4.2.1 Backend components

Each backend service encapsulates one domain. The principal components, their purpose and key dependencies:

Component	Purpose / function	Depends on
auth_service	Hash/verify passwords, register users, authenticate, issue & refresh JWT, KYC submit/review.	User, Wallet models; middleware
wallet_service	get balance, top_up, deduct, credit, list transactions — atomically updates the ledger.	Wallet, WalletTransaction
insurance_service	Manage product catalogue (CRUD, activate/deactivate).	InsuranceProduct
policy_service	Calculate premium, purchase policy, list/cancel policies.	Policy, Product, wallet_service, risk_engine
risk_engine	Compute event probability and premium; expose risk profile and model stats.	ml_models, RiskData
ml_models	FlightDelayModel & WeatherModel (load/predict_proba); persisted as .joblib.	scikit-learn / joblib
claims_engine	submit_manual_claim, submit_auto_claim, review_claim — orchestrates payout + notification.	Claim, Policy, wallet_service, notification_service
notification_service	Create and query user notifications.	Notification

4.2.2 API (router) components

Routers expose the services over a versioned REST API rooted at /api/v1. They validate input/output with Pydantic schemas and enforce authentication and role checks.

Router (prefix)	Actor	Main endpoints
/auth	Guest / User	register, login, refresh, me, kyc/submit, kyc/status
/wallet	User	GET /, topup, transactions
/insurance	Guest / User / Admin	list products, product detail, create, update, status (admin)

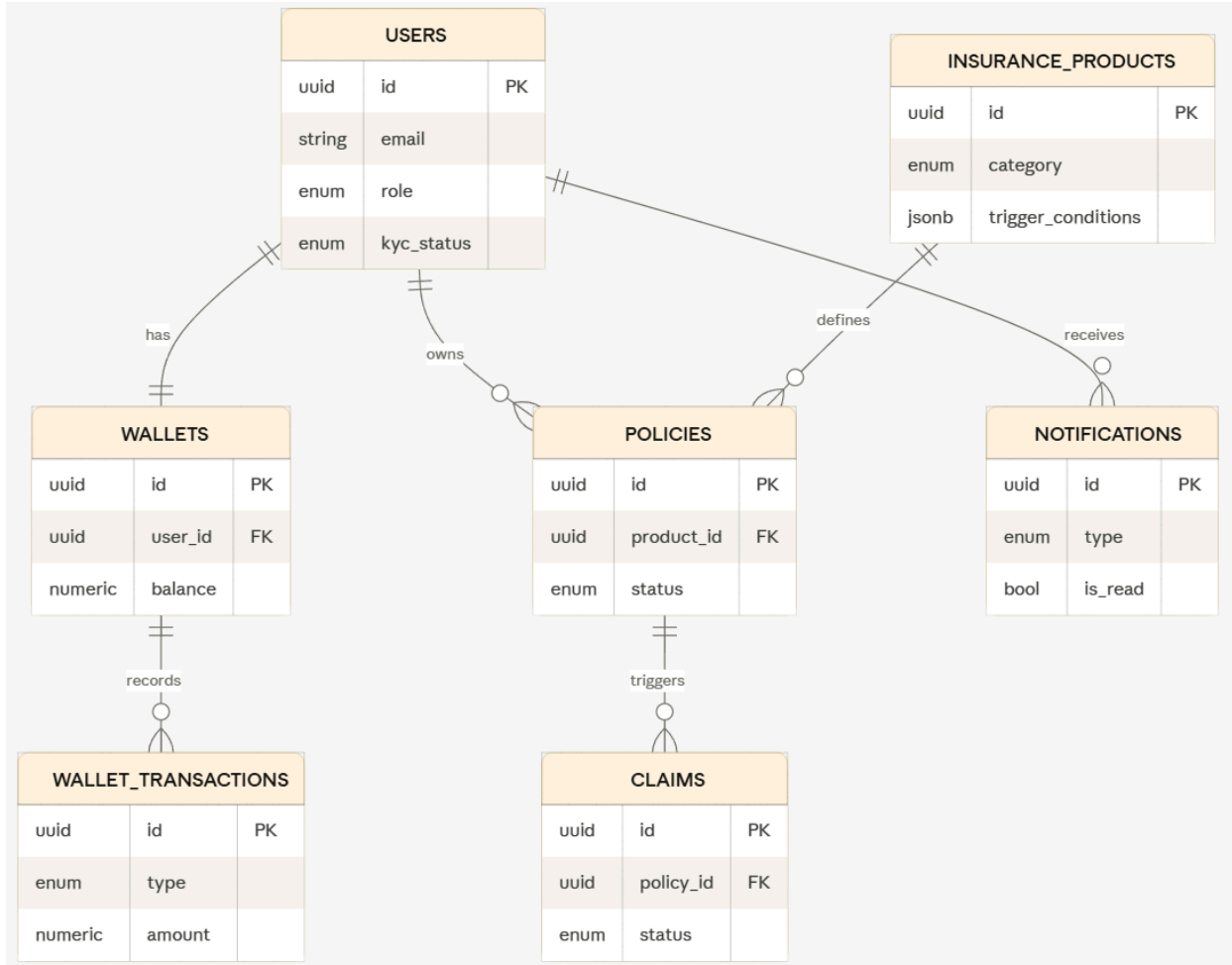
Router (prefix)	Actor	Main endpoints
/policies	User	calculate-premium, purchase, list, detail, cancel
/claims	User / System	submit, list, detail
/notifications	User	list, unread-count, read, read-all
/admin	Admin	dashboard, users, policies, claims, claims/review, kyc/pending, kyc review, ml stats, ml retrain

4.2.3 Data components (domain entities)

Persistent entities mapped by SQLAlchemy to PostgreSQL tables. UUID primary keys and timestamps are provided by shared mixins; JSONB columns store flexible schemas (parameters, trigger conditions, trigger data).

Entity (table)	Key attributes
User (users)	email, password_hash, full_name, role, phone_number, kyc_status, is_active
Wallet (wallets) / WalletTransaction	balance, currency; transaction: type, amount, balance_after, policy_id, claim_id
InsuranceProduct (insurance_products)	name, category, base_payout, min/max_duration_days, risk_margin, parameters_schema (JSONB), trigger_conditions (JSONB), is_active
Policy (policies)	user_id, product_id, premium_paid, payout_amount, status, parameters (JSONB), start_date, end_date
Claim (claims)	policy_id, trigger_type, trigger_event, trigger_data (JSONB), status, payout_amount, reviewed_by, processed_at
RiskData (risk_data)	product_category, region, event_date, event_type, event_severity, event_data (JSONB), source
SimulationSession	user_id, product_id, input_parameters (JSONB), triggers_activated (JSONB)
ChatSession	user_id, context_product_id, messages (JSONB)
Notification	user_id, type, title, message, is_read, reference_type, reference_id
ApiMonitorLog	api_name, endpoint, method, status_code, response_time_ms, error_message

Enumerations: UserRole (USER, ADMIN); KycStatus (NOT_SUBMITTED, PENDING, VERIFIED, REJECTED); TransactionType (TOP_UP, PREMIUM_PAYMENT, PAYOUT, REFUND); ProductCategory (FLIGHT_DELAY, CROP_WEATHER, GADGET, NATURAL_DISASTER, RAINFALL_EVENT); PolicyStatus (ACTIVE, EXPIRED, CLAIMED, CANCELLED); TriggerType (AUTOMATIC, MANUAL); ClaimStatus (PENDING, AUTO_APPROVED, MANUAL_REVIEW, APPROVED, PAID, REJECTED); NotificationType (CLAIM_TRIGGERED, PAYOUT_RECEIVED, POLICY_EXPIRING, POLICY_EXPIRED, SYSTEM)



4.2.4 Frontend components

- **User Web App:** views (Home, Login, Register, Kyc, Dashboard, InsuranceList/Detail, MyPolicies, Wallet, Notifications), reusable components (ProductCard, PremiumCalculator, RiskGauge, DynamicForm, BalanceCard, TopUpForm, TransactionList, NotificationBell), Pinia stores (auth, wallet, insurance, policy, notification) and an Axios service layer.
- **Admin Dashboard:** views (Login, Dashboard, Products, ProductForm, Policies, Claims, Users, KycReview), components (AdminHeader, AdminSidebar, StatsCard, ProductTable) and admin/insurance/claim services.

5. External Interfaces

5.1 Introduction

This section specifies how the AI-Powered Insurance Simulator communicates with human users, browser clients, internal API consumers, the database, and third-party software services.

The interfaces are derived from the SRS use cases and from the repository design in SPEC.md, README.md, CLAUDE.md, and the implementation plan.

All external communication follows a client/server model. Browser clients communicate only with the FastAPI backend. The backend is the sole component allowed to access PostgreSQL, external data providers, and the Gemini AI service. This preserves security boundaries, prevents API-key exposure, and keeps wallet and policy operations transactionally consistent.

5.2 User Interfaces

The system provides two responsive Vue 3 single-page applications: the User Web App and the Admin Dashboard. Both use route guards, Pinia stores, Axios API services, and role-aware navigation. Public screens are available to guests; protected screens require a valid JWT; administrative screens require the ADMIN role.

Interface	Primary actor	Design responsibility
Landing / Home	Guest, User	Presents educational content about parametric insurance, introduces the simulator concept, and provides entry points for registration, login, and product browsing.
Register / Login	Guest, User	Collects local credentials, validates input, submits authentication requests, stores tokens client-side, and redirects users according to role and KYC status.
KYC / Profile	User	Allows users to submit phone and identity details, view KYC status, and maintain profile information required before financial simulation operations.
Product List / Detail	Guest, User	Displays active insurance products, including description, category, base payout, duration limits, parameter schema, trigger conditions, and risk score.
Premium Calculator / Risk Gauge	User	Collects product-specific parameters, calls the risk engine through the policy API, and visualizes probability, risk multiplier, and final premium before purchase.
Simulation	User	Lets the user adjust scenario parameters, compare values with trigger thresholds, and observe whether a mock event would activate a payout.
Purchase / My Policies	User	Confirms purchase, shows wallet impact, lists active, expired, claimed, and cancelled policies, and links each policy to claims and transactions.
Wallet / Transactions	User	Shows SimCoin balance, permits simulated top-up after KYC verification, and provides an immutable transaction history.
Notifications	User	Shows claim, payout, policy lifecycle, and system notifications with unread count and read status.

Admin Dashboard	Admin	Displays operating metrics and provides navigation to product management, KYC review, policy/claim monitoring, risk analytics, and API health.
-----------------	-------	--

User-interface validation is performed on both sides: the client prevents incomplete forms and the backend remains authoritative through Pydantic schemas and database constraints. Error messages must be explicit enough for the user to understand failed operations such as invalid credentials, insufficient wallet balance, unverified KYC, inactive product, or rejected claim.

5.3 Internal REST API Interface

Both web clients consume the backend REST API under the base path `/api/v1`. Requests and responses are JSON. Protected requests include Authorization: Bearer `<access_token>`. Refresh-token requests use the refresh token issued at login. Routers validate payloads with Pydantic schemas and delegate all business rules to service components.

API group	Representative endpoints	Interface contract
/auth	POST /register, POST /login, POST /refresh, GET /me, POST /kyc/submit, GET /kyc/status	Public registration/login plus protected profile and KYC operations. Returns JWT tokens or typed user/KYC responses.
/wallet	GET /, POST /topup, GET /transactions	Requires USER role. Mutations require KYC VERIFIED and create WalletTransaction records with balance_after.
/insurance/products	GET /, GET /{id}, POST /, PUT /{id}, PATCH /{id}/status	Public/user read access for active products; ADMIN-only product creation, update, and activation status.
/policies	POST /calculate-premium, POST /purchase, GET /, GET /{id}, POST /{id}/cancel	Calculates premium through risk_engine and purchases policies atomically through policy_service and wallet_service.
/claims	POST /, GET /, GET /{id}	User-facing manual claim submission and claim listing. Automatic claim creation is invoked internally by the trigger monitor/claims engine.
/simulation	GET /products/{id}/config, POST /products/{id}/check-trigger, POST /products/{id}/log	Returns slider configuration, checks mock trigger activation, and stores SimulationSession records.

/chat	POST /message, GET /sessions, GET /sessions/{id}	Sends advisory prompts to Gemini through chatbot_service and persists ChatSession history.
/notifications	GET /, GET /unread-count, PATCH /{id}/read, PATCH /read-all	Lists and updates user notifications.
/admin	GET /dashboard, GET /users, GET /policies, GET /claims, PUT /claims/{id}/review, GET /kyc/pending, PATCH /kyc/{user_id}, GET /ml/model-stats	ADMIN-only monitoring, KYC, claim review, product/risk, and engine-health interfaces.

5.4 External System Interfaces

The backend integrates with external providers over HTTPS. API keys and service credentials are read from environment variables in the backend container and are never sent to browser clients. Each call should be wrapped with timeout, retry, and logging behaviour. The ApiMonitorLog entity records provider name, endpoint, status code, response time, response summary, and error details for the admin monitoring view.

External system	Direction / protocol	Purpose and exchanged data
OpenWeatherMap	Outbound REST/JSON	Provides rainfall, weather severity, region, and timestamp data used for crop/weather, rainfall-event, and disaster trigger evaluation.
AviationStack	Outbound REST/JSON	Provides flight status, airport/route data, scheduled time, actual time, and delay duration for flight-delay policy triggers.
Google Gemini 2.0 Flash	Outbound REST/JSON	Generates advisory chatbot responses from user question, product catalogue context, policy context, and risk statistics. The chatbot is advisory only and cannot execute purchases or wallet operations.
Mock trigger provider	Internal/demo REST or service call	Supplies realistic simulated event payloads when real API keys are unavailable or when a user runs simulation mode.

Interface concern	Design rule	SRS rationale
API key handling	Keys are stored in backend environment variables or deployment secrets and are excluded from frontend bundles.	External API keys must not be exposed to the client side.

Retry and timeout	Provider calls use bounded timeouts and retry failed transient responses before recording failure.	Detect External Event requires a retry mechanism for failed external API calls.
Payload validation	Incoming provider payloads are normalized and validated before trigger evaluation.	Process Claim Automatically depends on objective trigger conditions.
Monitoring	Every provider call produces an ApiMonitorLog entry for dashboard review.	System errors and important events must be logged.
Fallback mode	Simulation/mock providers can be used for demos and testing without real external dependency availability.	The simulator must remain usable for educational scenarios.

5.5 Database Interface

The database interface is internal to the backend. Routers and frontend code never access PostgreSQL directly. SQLAlchemy models define the object interface, Alembic manages schema evolution, and service methods own transaction boundaries.

Database interface	Design decision	Affected components
Connection management	FastAPI dependencies provide a SQLAlchemy session per request; service functions receive the session explicitly.	All routers and services.
Transactions	Wallet credit, wallet deduction, policy purchase, claim payout, and refund operations run as atomic units and roll back on failure.	wallet_service, policy_service, claims_engine.
Locking	Wallet mutations acquire row-level locks where supported to prevent concurrent balance races.	wallet_service.deduct, wallet_service.credit, wallet_service.top_up.
Audit ledger	WalletTransaction is append-only by convention and records transaction type, amount, balance_after, policy_id, and claim_id.	Wallet history, purchase audit, payout audit.
Flexible product rules	InsuranceProduct.parameters_schema and trigger_conditions are JSONB documents so admins can define product-specific forms and parametric thresholds without schema changes.	insurance_service, policy_service, simulation_engine, claims_engine.

5.6 Communication, Security, and Error Interface

The API returns HTTP status codes consistently: 200 for successful reads, 201 for created resources, 400 for invalid business requests, 401 for missing/invalid tokens, 403 for insufficient role or KYC status, 404 for missing resources, 409 for conflicts such as duplicate email or duplicate claims, and 500 only for unexpected server failures.

Cross-origin access is limited to configured frontend origins. Passwords are hashed before persistence. JWT middleware extracts the current user and exposes role-check dependencies such as `require_admin`. Administrative operations are never authorized only by client-side routing; the backend enforces RBAC for every protected API.

6. Detailed Design

6.1 Introduction

This section describes the internal design of the principal backend and frontend-facing components. The design follows the Router-Service-Model decomposition established in Section 4. Routers own HTTP contracts, services own business rules and transaction orchestration, and models own persistence structure.

The SRS emphasizes three critical qualities: secure access control, data consistency for virtual-financial operations, and rapid automated claim settlement. Therefore the most detailed design is provided for authentication, wallet, policy purchase, risk/simulation, claims, external triggers, chatbot, notifications, and administration.

6.2 Component - Authentication and KYC Service (`auth_service`)

The authentication service provides account registration, login, refresh-token handling, current-user lookup, KYC submission, and administrative KYC review. It uses bcrypt-style password hashing and never stores raw passwords. JWT tokens carry user identity and role claims, while account and KYC status remain authoritative in the database.

Operation	Input	Output / effect
register	email, password, full_name, optional phone	Creates User with role USER and KYC NOT_SUBMITTED; creates Wallet with initial SimCoin balance.
authenticate	email, password	Verifies active account and password hash; returns access and refresh tokens.
refresh_access_token	refresh token	Validates refresh token and issues a new access token.
get_current_user	Bearer access token	Returns authenticated User or raises 401.
submit_kyc	phone and identity details	Sets KYC status to PENDING and stores submitted metadata.
review_kyc	admin decision and optional rejection reason	Sets KYC status to VERIFIED or REJECTED.

Registration flow: validate the request schema; reject duplicate email; hash password; create the user and wallet in one database transaction; return the user response. Login flow: validate credentials; reject inactive accounts; issue tokens; the frontend stores tokens and fetches /auth/me to build the role-aware navigation menu.

6.3 Component - Wallet Service (wallet_service)

The wallet service owns all SimCoin balance changes. It is the only component allowed to mutate wallet balances directly. Every mutation creates a WalletTransaction record that captures type, amount, balance_after, and references to the policy or claim that caused the mutation.

Operation	Rules	Failure handling
get_balance	Returns wallet id, balance, currency, and timestamps.	404 if the wallet is missing.
top_up	Requires USER role and KYC VERIFIED; adds simulated SimCoin.	403 if KYC is not verified; rollback on database failure.
deduct	Checks sufficient balance; subtracts premium; records PREMIUM_PAYMENT transaction.	400 if balance would become negative; rollback preserves original balance.
credit	Adds payout or refund; records PAYOUT or REFUND transaction.	Rollback if transaction record cannot be written.
get_transactions	Returns paginated newest-first ledger entries.	Never deletes or rewrites historical transactions.

Concurrency design: mutating operations should lock the wallet row before reading the balance, calculate the new balance in memory, write both wallet and transaction rows, then commit. This prevents two simultaneous purchases from spending the same balance.

6.4 Component - Insurance Product Service (insurance_service)

The insurance product service controls the marketplace catalogue. Products are micro-insurance definitions with category, description, payout, duration boundaries, risk margin, parameter schema, trigger conditions, and activation status.

Design element	Description	SRS relation
parameters_schema	JSONB form definition used by the frontend DynamicForm and simulation sliders.	View product details; run simulation.
trigger_conditions	JSONB thresholds such as delay_minutes >= 120 or rainfall_mm >= 50.	Trigger Condition; Process Claim Automatically.
risk_margin	Multiplier applied during premium calculation to keep virtual products financially realistic.	AI-driven risk and claim simulation.
is_active	Controls whether a product is visible and purchasable.	Admin policy creation and deployment.

created_by	Links product configuration to the administrator account.	Manage insurance products.
------------	---	----------------------------

Admin create/update flow: validate product metadata and JSON rule structure; persist the product; return the updated catalogue item. Public/user list flow returns active products and computed risk scores; inactive products remain available only to admin views.

6.5 Component - Policy Purchase Service (policy_service)

The policy service implements the purchase lifecycle described in UC006. It coordinates product lookup, KYC enforcement, premium calculation, wallet deduction, policy creation, cancellation, and expiration.

Step	Purchase behaviour	Consistency rule
1	Validate that the user is authenticated and KYC VERIFIED.	Reject before any wallet mutation if the user is not eligible.
2	Load the active product and validate user parameters against the product schema.	Reject inactive or malformed product requests.
3	Call risk_engine.calculate_premium to obtain event_probability, risk_multiplier, and premium.	Premium preview and purchase use the same calculation source.
4	Call wallet_service.deduct for the exact premium.	Wallet deduction and policy creation are part of one logical transaction.
5	Create Policy with ACTIVE status, premium_paid, payout_amount, parameters, start_date, and end_date.	Rollback if policy persistence fails after deduction.
6	Return policy response and allow UI to refresh wallet and policy list.	User sees immediate feedback after purchase.

Cancellation flow: verify policy ownership and ACTIVE status; calculate the refundable amount according to the product/cancellation rule; credit the wallet through wallet_service.credit; set policy status to CANCELLED. Expiration flow marks policies as EXPIRED when end_date passes without a claim.

6.6 Component - Risk and ML Engine (risk_engine + ml_models)

The risk engine estimates event probability and premium. It uses lightweight classification models such as FlightDelayModel and WeatherModel, stored as joblib artifacts, and may fall back to historical RiskData statistics when model artifacts are missing.

Input	Processing	Output
Product category	Select model or statistical profile for flight, weather, disaster, rainfall, or manual product.	Model identifier and supported feature set.

Scenario parameters	Normalize fields such as route, month, region, rainfall, severity, or delay threshold.	Feature vector for prediction.
Historical risk data	Compute category, region, season, and event-frequency statistics.	Risk profile for admin analytics and chatbot context.
Base payout and risk margin	Expected loss = probability x payout; premium = expected loss x (1 + risk_margin), with minimum/maximum bounds when configured.	event_probability, risk_multiplier, final_premium, risk_level.

The premium calculation must complete within the SRS two-second API target under normal load. Model loading should happen at service startup or be cached by a singleton accessor so request handlers do not repeatedly load joblib files.

6.7 Component - Simulation Engine

The simulation engine provides the educational mode from UC008. It is not a real claim path; instead, it lets users manipulate scenario values and immediately see whether the trigger would activate. This supports the SRS goal of helping users understand parametric insurance before spending virtual currency.

Operation	Design behaviour	Stored data
get_config	Read product parameters_schema and trigger_conditions to produce slider ranges, labels, threshold bars, and explanatory text.	No mutation.
check_trigger	Compare submitted values with trigger rules and calculate mock payout amount when triggered.	Optional transient result.
log_session	Persist user id, product id, input_parameters, and triggers_activated.	SimulationSession record for analytics.

Simulation should use the same trigger definitions as the claims engine so that user education remains consistent with real automatic settlement behaviour. When the user selects mock payout, the UI may display a simulated confirmation, but it must not credit the real wallet unless the flow is explicitly a demo transaction.

6.8 Component - Parametric Claims Engine (claims_engine)

The claims engine is the core differentiator of the platform. It supports automatic parametric claims for objective trigger events and manual claims for non-parametric products such as gadget coverage.

Path	Trigger	Processing result
Automatic claim	External event payload matches an ACTIVE policy's trigger_conditions.	Create Claim with AUTOMATIC trigger, approve/pay it, credit wallet, mark policy CLAIMED, notify user.

No-trigger event	External event payload does not meet threshold.	Record monitoring outcome if needed; do not create a claim.
Manual claim	User submits evidence for a product that requires review.	Create Claim with MANUAL trigger and MANUAL_REVIEW/PENDING status.
Admin approval	Admin approves manual claim.	Credit wallet, set claim PAID, mark related policy CLAIMED, notify user.
Admin rejection	Admin rejects manual claim.	Set claim REJECTED with reason; no wallet mutation.

Automatic claim algorithm: receive normalized provider payload; validate required fields; find active policies affected by route, region, time window, or product category; evaluate each JSON trigger expression; skip policies already claimed; create claim; credit wallet inside the same transactional unit; update policy and claim status; emit payout notification. The end-to-end target after receiving provider data is less than five seconds.

6.9 Component - External Trigger Monitor

The trigger monitor periodically calls external providers or mock providers, records ApiMonitorLog entries, and invokes the claims engine when a relevant event is detected. It is designed as a backend background job so browser clients do not need to stay connected for claims to settle.

Design concern	Decision	Reason
Polling schedule	Run periodically through a scheduler or cron-like background task.	Meets automated settlement without manual claim submission.
Provider adapters	Separate adapter functions normalize OpenWeatherMap, AviationStack, and mock payloads.	Keeps claims_engine independent of third-party response formats.
Idempotency	Use policy status and existing claim checks to avoid duplicate payouts.	Prevents multiple credits for the same event.
Observability	Log success, latency, response summary, and errors to ApiMonitorLog.	Supports admin monitoring and debugging.
Retry	Retry transient failures with bounded attempts, then surface degraded status.	Satisfies reliability requirement for external API calls.

6.10 Component - AI Chatbot Service

The chatbot service sends user questions to Gemini with controlled context: product catalogue summary, relevant user policy context, risk statistics, and a system instruction that the chatbot is advisory only. It persists conversations in ChatSession so the user can review advice and the system can maintain context.

Step	Chatbot behaviour	Boundary
1	Validate authenticated user and message length.	Reject empty or excessive prompts.
2	Build context from products, optional selected product, risk profile, and recent chat history.	Do not include secrets or unnecessary personal data.
3	Call Gemini API with timeout and fallback handling.	Return a graceful message if the provider is unavailable.
4	Persist user message and assistant response in ChatSession.	Chatbot cannot purchase policies, alter wallet, or approve claims.
5	Return response to client within the SRS target of less than three seconds under normal conditions.	Frontend shows loading and error states.

6.11 Component - Notification Service

The notification service creates short user-facing records for important events. Notifications decouple backend events from UI display: a claim payout, policy expiry, KYC result, or system announcement can be created by services and later displayed by the frontend notification bell.

Notification type	Created by	Example event
CLAIM_TRIGGERED	claims_engine	External data matched a policy trigger.
PAYOUT_RECEIVED	wallet_service / claims_engine	A payout was credited to the wallet.
POLICY_EXPIRING	policy expiration job	A policy is within 24 hours of expiry.
POLICY_EXPIRED	policy expiration job	A policy ended without a claim.
SYSTEM	admin or backend service	KYC decision, maintenance message, or general announcement.

6.12 Component - Admin and Monitoring

The admin subsystem provides operational visibility and controlled configuration. It aggregates metrics from users, products, policies, claims, risk data, and API monitor logs. Every admin endpoint requires ADMIN role on the backend.

Admin function	Data source	Design behaviour
Dashboard metrics	User, Policy, Claim, WalletTransaction	Show total users, active policies, paid claims, total payout, and recent activity.
KYC review	User KYC fields	List PENDING users and approve/reject with optional reason.
Product management	InsuranceProduct	Create/update/deactivate products and rule definitions.

Claims monitoring	Claim, Policy, WalletTransaction	Filter claims by status, product, user, and date; review manual claims.
Risk analytics	RiskData, ml_models	Expose model availability, event distributions, risk profiles, and retraining actions.
API monitoring	ApiMonitorLog	Display provider health, latency, recent failures, and last successful check.

6.13 Cross-Cutting Design Decisions

Concern	Design decision	Affected requirement
Authentication	JWT access/refresh token model with backend-enforced RBAC.	All protected APIs require valid token; Admin/User separation.
Password security	Hash passwords before storing; never return password_hash in responses.	Security and access control.
Financial consistency	Wallet mutations and related policy/claim records commit or roll back together.	ACID for purchase and payout.
Performance	Use cached ML models, indexed queries, paginated lists, and concise JSON payloads.	2-second API response target and scalability.
External reliability	Retries, timeouts, fallback mock data, and ApiMonitorLog entries.	Reliable external event detection.
Scalability	Stateless FastAPI backend and separate Docker services.	Horizontal scalability using containerization.
Usability	Clear success/failure messages and role-aware navigation.	Simple and intuitive UI feedback.
Auditability	Append-only transaction ledger and claim/policy status history.	Debugging, monitoring, and transaction traceability.

7. Annexes

7.1 Requirements Traceability Matrix

The matrix links SRS use cases to the design components that realise them. It can be used by developers to locate implementation responsibility and by testers to derive functional test cases.

UC	Use case	Realising design elements
UC001	Register Account	/auth/register, auth_service.register, User, Wallet.

UC002	Login	/auth/login, /auth/refresh, auth_service.authenticate, JWT middleware.
UC003	Browse Platform Information	User Web App Home view and public routing.
UC004	View Insurance Products	/insurance/products, insurance_service.get_products, ProductCard.
UC005	View Insurance Product Details	/insurance/products/{id}, InsuranceDetailView, DynamicForm, RiskGauge.
UC006	Purchase Insurance Policy	/policies/purchase, policy_service, risk_engine, wallet_service.deduct, Policy.
UC007	View Purchased Policies	/policies, /policies/{id}, policy_service.get_user_policies, MyPoliciesView.
UC008	Run Insurance Simulation	/simulation endpoints, simulation_engine, risk_engine, SimulationSession, Simulation UI.
UC009	AI Chatbot Consultation	/chat/message, chatbot_service, Gemini integration, ChatSession.
UC010	Detect External Event	trigger_monitor, provider adapters, ApiMonitorLog.
UC011	Process Claim Automatically	claims_engine.submit_auto_claim, Policy, Claim, trigger_conditions.
UC012	Credit Wallet After Claim	wallet_service.credit, WalletTransaction, notification_service.
UC013	View Transaction History	/wallet/transactions, WalletTransaction, TransactionList.
UC014	Manage Insurance Products	Admin product views, /insurance/products admin endpoints, insurance_service.
UC015	Manage Product Rules	ProductForm, parameters_schema, trigger_conditions, insurance_service.
UC016	Monitor Claims	Admin Claims view, /admin/claims, /admin/claims/{id}/review, claims_engine.
UC017	Manage Profile	/auth/me, user profile/KYC views, auth_service.
UC018	Manage User Accounts	/admin/users, /admin/kyc/pending, /admin/kyc/{user_id}, auth_service.review_kyc.
UC019	Monitor Parametric Engine	/admin dashboard, /admin/ml/model-stats, ApiMonitorLog, trigger_monitor.

7.2 Non-Functional Requirement Coverage

SRS requirement	Design mechanism	Verification approach
API response time <= 2 seconds for login, product view, simulation.	Indexed queries, cached ML models, concise Pydantic responses, pagination.	Measure endpoint latency with backend tests and

		browser/network inspection.
Automated claim processing <= 5 seconds after external data is received.	Background trigger monitor invokes claims_engine directly; payout transaction is local database work.	Mock provider event test measuring event receipt to PAID claim status.
Chatbot response <= 3 seconds under normal conditions.	Timeout-bound Gemini call, compact context, graceful fallback.	Integration test with mock Gemini and runtime timing logs.
Passwords securely hashed.	auth_service hashes passwords and schemas never expose password_hash.	Unit test registration and inspect stored user record.
Protected APIs require valid token.	JWT middleware and route dependencies enforce current user and require_admin.	401/403 route tests.
External API keys not exposed client-side.	Keys stored in backend environment variables only.	Frontend bundle review and config inspection.
ACID for policy purchase and wallet payout.	Service-level transaction boundaries and rollback on failure.	Failure-injection tests for purchase and payout.
Retry mechanism for failed external API calls.	Provider adapters apply bounded retry and write ApiMonitorLog.	Mock transient provider failures.
Clear success and failure feedback.	Frontend toast/error states plus descriptive backend error messages.	Manual UI acceptance tests.
Horizontal scalability through containerization.	Docker Compose separates database, backend, user frontend, and admin frontend.	Container startup and multiple backend replica smoke test.

7.3 Endpoint-to-Component Mapping

Endpoint group	Router	Primary service	Primary models
/api/v1/auth	auth.py	auth_service	User, Wallet.
/api/v1/wallet	wallet.py	wallet_service	Wallet, WalletTransaction.
/api/v1/insurance/products	insurance.py	insurance_service	InsuranceProduct, RiskData.
/api/v1/policies	policies.py	policy_service, risk_engine, wallet_service	Policy, InsuranceProduct, WalletTransaction.

/api/v1/claims	claims.py	claims_engine, notification_service	Claim, Policy, WalletTransaction, Notification.
/api/v1/simulation	simulation.py	simulation_engine, risk_engine	SimulationSession, InsuranceProduct, RiskData.
/api/v1/chat	chatbot.py	chatbot_service	ChatSession, InsuranceProduct.
/api/v1/notifications	notifications.py	notification_service	Notification.
/api/v1/admin	admin.py	auth_service, claims_engine, insurance_service, risk_engine	User, Policy, Claim, InsuranceProduct, ApiMonitorLog.

7.4 Data and Relationship Summary

The ERD in docs/ERD.md defines the persistent model. The central relationships are: one User has one Wallet; one User purchases many Policies; one InsuranceProduct can be purchased as many Policies; one Policy can produce Claims; one Wallet owns many WalletTransaction records; Notifications, ChatSessions, and SimulationSessions belong to Users; ApiMonitorLog is standalone monitoring data.

Entity	Key design role	Important integrity rule
User	Identity, role, KYC, and account lifecycle.	email is unique; inactive users cannot authenticate.
Wallet	Current simulated balance.	balance must never be negative; one wallet per user.
WalletTransaction	Immutable audit trail for top-up, premium, payout, refund.	References policy_id or claim_id when applicable; append-only by convention.
InsuranceProduct	Product catalogue and rule definition.	trigger_conditions and parameters_schema must be valid JSON structures.
Policy	Purchased virtual contract.	Lifecycle status controls whether triggers can produce claims.
Claim	Automatic or manual claim record.	Paid claim must correspond to wallet payout transaction.
RiskData	Historical event data for risk and model analytics.	Indexed by category, region, and date.
ApiMonitorLog	External provider observability.	Records both successful and failed provider calls.

7.5 IEEE 1016 and UML Mapping

The architectural view in Section 3 maps to deployment and component diagrams. The decomposition view in Section 4 maps to package/class diagrams and the ERD. The interface view in Section 5 maps to UI wireframes, API tables, and external interface contracts. The detailed behavioural view in Section 6 maps to sequence and activity diagrams for registration, purchase, simulation, and automatic claims. Together these views provide the structural and behavioural coverage recommended by IEEE 1016.

Recommended diagram	Source section	Purpose
Deployment diagram	3.2 System Architecture	Shows Docker services: user frontend, admin frontend, backend, PostgreSQL, and external providers.
Component diagram	4.2 Decomposition Description	Shows routers, services, models, and frontend modules.
ERD / class diagram	4.2.3 and 7.4	Shows persistent entities, cardinalities, constraints, and key attributes.
Sequence diagram - Register/Login	6.2	Shows user, frontend, auth router, auth service, database, and wallet creation.
Sequence diagram - Purchase Policy	6.5	Shows product lookup, premium calculation, wallet deduction, and policy creation.
Activity diagram - Automatic Claim	6.8 and 6.9	Shows external event detection, trigger evaluation, claim creation, payout, and notification.
Activity diagram - Simulation	6.7	Shows scenario input, trigger checking, result display, optional mock payout, and session logging.